

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра Систем информатики

Направление подготовки 09.03.01 – Информатика и вычислительная техника

ОТЧЕТ

Обучающегося Якутиной Светланы Александровны группы № 22215 курса 4
(Ф.И.О. полностью)

Тема задания: Разработка репозитория для редактирования онтологии предметной области

Новосибирск 2025

Оглавление

Введение	3
1 Реализация класса <code>OntologyRepository</code>	4
В процессе запуска функции, приведенной на листинге 1, происходит выполнение запроса к базе Neo4j для поиска всех родительских классов указанного класса. Метод возвращает список словарей, каждый из которых содержит информацию о родительском классе, включая его <code>uri</code> и свойства.	5
Заключение	6

Введение

Целью данной работы является разработка Python-класса для работы с онтологией в графовой базе данных Neo4j.

Для реализации цели были поставлены следующие задачи:

- a. Разработка класса `OntologyRepository`, реализующего доступ к сущностям онтологии (классам, объектам и их атрибутам).
- b. Реализация методов для создания, обновления, удаления и получения классов, объектов и их свойств.
- c. Формирование структурных характеристик классов для оценки их связей и свойств внутри онтологии.

1 Реализация класса `OntologyRepository`

- **Методы получения онтологии**

- `get_ontology()`
Возвращает полную онтологию, включая все классы, атрибуты и связи между ними. Используется для визуализации или анализа структуры онтологии.
- `get_ontology_parent_classes()`
Возвращает классы, не имеющие родителей. Позволяет определить корневые элементы онтологии.

- **Работа с классами**

- `get_class(class_uri)`
Получает информацию о конкретном классе по его URI.
- `get_class_parents(class_uri)`
Возвращает список родительских классов для указанного класса.
- `get_class_children(class_uri)`
Возвращает список дочерних классов для указанного класса.
- `get_class_objects(class_uri)`
Возвращает все объекты, являющиеся экземплярами указанного класса.
- `update_class(class_uri, title, description)`
Обновляет свойства класса, такие как `title` и `description`.
- `create_class(title, description, parent_uri=None)`
Создает новый класс и, при указании `parent_uri`, связывает его с родительским классом через отношение `SUBCLASS_OF`.
- `delete_class(class_uri)`
Удаляет указанный класс, его подклассы и объекты, которые являются его экземплярами.

- **Атрибуты классов**

- `add_class_attribute(class_uri, title)`
Добавляет атрибут типа `DatatypeProperty` к классу. Атрибут связывается с классом через отношение `DOMAIN`.
- `delete_class_attribute(property_uri)`
Удаляет атрибут класса по uri.
- `add_class_object_attribute(class_uri, attr_name, range_class_uri)`
Добавляет объектный атрибут (`ObjectProperty`) к классу. Связывает его с классом через `DOMAIN` и с целевым классом через `RANGE`.
- `delete_class_object_attribute(object_property_uri)`
Удаляет объектный атрибут по URI.
- `add_class_parent(parent_uri, target_uri)`
Добавляет родительский класс для указанного класса через отношение `SUBCLASS_OF`.

- **Работа с объектами**

- `get_object(object_uri)`
Получает объект по его uri.
- `delete_object(object_uri)`
Удаляет объект по URI.
- `create_object(class_uri, props)`
Создает объект, относящийся к указанному классу, с заданными свойствами.
- `update_object(object_uri, props)`
Обновляет свойства объекта.

- **Получение атрибутов класса и объектных свойств**

- `collect_signature(class_uri)`
Возвращает список атрибутов класса (`DatatypeProperty`) и объектных свойств (`ObjectProperty`) с указанием направления связи.
Используется для анализа структуры класса и его связей в онтологии.

Для реализации алгоритма `OntologyRepository` был написан код, приведенный на листинге 1.

```
def get_class_parents(self, class_uri: str) -> List[Dict[str, Any]]:
    query = """
MATCH      (c:Class      {uri:      $uri})-[:SUBCLASS_OF]->(p:Class)
RETURN
    """
    return self.repo.run_custom_query(query, {"uri": class_uri})
```

Листинг 1 – пример реализации метода `get_class_parents`

В процессе запуска функции, приведенной на листинге 1, происходит выполнение запроса к базе Neo4j для поиска всех родительских классов указанного класса. Метод возвращает список словарей, каждый из которых содержит информацию о родительском классе, включая его uri и свойства.

Заключение

В результате проведенной работы был реализован класс `OntologyRepository`, позволяющий:

1. Получать полную онтологию и её структуру.
2. Управлять классами и их атрибутами (создание, обновление, удаление).
3. Работать с объектами онтологии.
4. Получить атрибуты класса и объектные свойства для анализа их свойств и связей.

Все поставленные задачи из введения выполнены, что позволяет использовать данный класс в проектах по управлению онтологиями в Neo4j.